

BLADE Complete Framework Manual

Structure Generation, Thermodynamic Fitting, Phase Visualization, Database Construction,
and Oxidation Analysis

BLADE Project

July 18, 2026

Contents

1	Purpose and Scope	3
2	Repository Layout	3
3	Installation	4
3.1	Python and Package Managers	4
3.1.1	Pixi	4
3.1.2	uv	4
3.2	External Programs	4
4	End-to-End Orchestrator	5
4.1	Commands	5
4.2	Major Stage Switches	5
4.3	Configuration Injection	5
5	Complete TOML Reference	6
5.1	[paths]	6
5.2	[tdb]	6
5.3	[tdb_fit], [tdb_sqs], and [tdb_scraps]	6
5.4	[phase]	7
5.5	[tdb_inputs]	7
5.6	[phase_plots] and [phase_grid]	8
5.7	[database]	8
5.8	[oxidation] and Modes	8
6	Structure and TDB Calculations	9
6.1	Composition Enumeration	9

6.2	SQS Generation	9
6.3	MLIP Relaxation and TDB Fit	9
7	Phase Visualization	9
8	Oxidation Database Workflow	10
8.1	Composition List	10
8.2	Reference Structures and Energies	10
8.3	Database Assembly	10
9	Oxidation Equilibrium and Maps	10
9.1	Public API	10
9.2	Equilibrium Model	10
9.3	Region Identity and Labels	11
9.4	Outputs	11
10	Caching, Restarting, and Storage	11
11	Every Example Script	12
12	Library Class and Module Reference	13
12.1	blade.tools	13
12.2	blade.analysis	13
12.3	blade.oxidation	13
12.4	blade.oxidation.framework	13
13	Reproducible Execution Records	14
14	Quality Checks	14
15	Troubleshooting	15
16	Extending to a New Material System	15
17	Standalone Versus Configured Execution	16

1 Purpose and Scope

BLADE is a general, high-throughput materials workflow that connects chemical system enumeration, prototype-based structure generation, special quasirandom structures (SQS), machine-learned interatomic-potential (MLIP) relaxation, CALPHAD database fitting, thermodynamic visualization, reference database construction, and environmental stability or oxidation analysis.

The example configurations use diboride-like and MAX-like structures to demonstrate prototype definitions. A phase is defined by lattice vectors, coordinates, variable and fixed sublattice labels, site species, and optional fixed compositions. Fixed species such as B, C, or any other element are specified through the user configuration.

The framework’s practical novelty is the reproducible connection of:

1. arbitrary prototype and sublattice definitions;
2. high-throughput SQS generation through ATAT/mcsqs or SCRAPS;
3. calculator-selectable structure relaxation and energy generation;
4. automated fitting of per-system thermodynamic databases;
5. binary through higher-order phase visualization;
6. a shared energy database for environmental equilibrium calculations;
7. static, animated, and exact-hover visual outputs; and
8. cache-aware restart behavior across expensive stages.

This is a workflow and software manual, not a validation claim for a specific potential, fitted database, phase set, or scientific conclusion. Scientific accuracy remains conditional on structures, energies, references, fitting choices, solver assumptions, and convergence.

2 Repository Layout

Path	Purpose
src/blade/tools/	Composition, SQS, SCRAPS, cutoff, and TDB-generation classes.
src/blade/analysis/	Structural data extraction and thermodynamic/structure visualization.
src/blade/oxidation/	Database construction plus public oxidation calculation and visualization APIs.
src/blade/oxidation/framework/	General equilibrium, map, batch, single-composition, labeling, caching, and plotting implementation.
examples/full_framework.py	TOML-driven end-to-end orchestrator.
examples/full_framework.toml	Standard complete-workflow demonstration configuration.
examples/full_framework_max.toml	Multibasis demonstration configuration.
examples/structures/	Direct structure/SQS/TDB examples and complete prototype drivers.
examples/oxidation/	Direct database and oxidation examples.
examples/extra/	Optional refitting, comparison, and combined phase visualization utilities.
Files/	User-configurable persistent inputs, caches, databases, and outputs.

Path	Purpose
<code>uv.lock</code>	Reproducible uv dependency resolution.
<code>pixi.toml</code> and <code>pixi.lock</code>	Pixi environment and task definitions.

3 Installation

3.1 Python and Package Managers

BLADE targets Python 3.12. Use either Pixi or uv.

3.1.1 Pixi

```
cd /path/to/BLADE
pixi install
pixi run python -c "import blade; print(blade.__version__)"
pixi run check
```

3.1.2 uv

```
cd /path/to/BLADE
uv sync --locked --extra workflow
uv run --locked --extra workflow python -c "import blade; print(blade.__version__)"
uv run --locked --extra workflow python examples/full_framework.py --check
```

The `workflow` extra installs `MaterialsFramework`, `pycalphad`, `mp-api`, and the configured MLIP dependency. The lock file should be committed and retained with a run.

3.2 External Programs

Program	Requirement
ATAT	Supplies <code>sqs2tdb</code> , <code>corrdump</code> , and <code>mcsqs</code> ; required by the <code>mcsqs</code> SQS/fitting path.
SCRAPS	Required only when <code>tdb.sqs_method="scraps"</code> . Configure <code>paths.scraps_repo</code> .
FFmpeg	Required for MP4 movies and synchronized phase-grid video output.
Materials Project	Network access and an API key are required only for uncached reference downloads.
MLIP runtime	Calculator-dependent models, device support, and model files must be available.

Store the Materials Project credential in the environment:

```
export MP_API_KEY="your-key"
```

Do not commit credentials to TOML, source, logs, or documentation.

4 End-to-End Orchestrator

4.1 Commands

The default input is `examples/full_framework.toml`.

```
# Validate files, settings, imports, and external dependencies.
pixi run python examples/full_framework.py --check

# Print enabled stages without executing calculations.
pixi run python examples/full_framework.py --dry-run

# Run the default configuration.
pixi run python examples/full_framework.py

# Run an alternate configuration.
pixi run python examples/full_framework.py examples/full_framework_max.toml
```

Equivalent uv commands prepend `uvrun--locked--extraworkflow` instead of `pixirun`.

4.2 Major Stage Switches

The top of each TOML exposes the high-level controls:

```
[stages]
sqs_generation = true
tdb_fitting = true
phase_diagrams = true
oxide_database = true
oxidation_graphs = true
```

The orchestrator executes these internal groups in order:

1. `tdb`: SQS generation and/or TDB fitting;
2. `phase_visualization`: phase plots/GIFs and grid movies;
3. `database`: reference structures, energies, and phase database;
4. `oxidation`: batch or single-composition analysis.

The two first booleans both route through the selected TDB driver. The runner injects independent `run_sqs` and `run_tdb` values, so either half can be run alone. The remaining booleans enable their corresponding stages.

4.3 Configuration Injection

`full_framework.py` parses the selected TOML with the Python standard library. For legacy direct examples, it parses the target script's abstract syntax tree, removes only top-level assignments whose names are supplied by the orchestrator, and executes the script with injected values. Functions and workflow code remain the same as direct execution. Database and oxidation stages use class APIs directly.

5 Complete TOML Reference

5.1 [paths]

Setting	Meaning
<code>blade_root</code>	Repository root used to find <code>src/</code> and <code>examples/</code> .
<code>files_dir</code>	Persistent BLADE data/output root.
<code>sqsdb_dir</code>	ATAT SQS database location used by the fitting tools.
<code>scraps_repo</code>	SCRAPS checkout; used only by the SCRAPS backend.

5.2 [tdb]

Setting	Meaning
<code>sqs_method</code>	"mcsqs" or "scraps".
<code>prototype_driver</code>	"standard" or "multibasis"; selects the bundled general driver implementation.
<code>level</code>	SQS composition level to generate/fit.
<code>skip_existing_sqs</code>	Reuse completed SQS artifacts.
<code>skip_existing_tdb</code>	Reuse existing TDB output.
<code>refit_existing_tdb</code>	Refit from retained structures/energies even when SQS generation is skipped.
<code>skip_existing_plots</code>	Reuse each plot independently when that output exists.
<code>generate_gibbs_energy</code>	Enable per-binary Gibbs-energy plot.
<code>generate_gibbs_mixing</code>	Enable per-binary Gibbs-mixing plot.
<code>generate_phase_diagram</code>	Enable applicable phase-diagram plot.
<code>generate_combined_phase_diagram</code>	Enable montage/combined diagram output.
<code>generate_contcar_plots</code>	Enable combined structure images.
<code>run_movie</code>	Enable relaxation movie generation.
<code>mlip</code>	MaterialsFramework calculator registry name.
<code>primary_elements, secondary_elements</code>	Element pools used for system enumeration.
<code>primary_min/max, secondary_min/max</code>	Number of elements drawn from each pool per system.
<code>liquid</code>	Include configured liquid treatment in fitting.
<code>comps_folder</code>	Per-system output directory below <code>Files/</code> .

[`tdb_mlip_kwargs`] is passed to the selected calculator, for example `steps` and `device`. Calculator-specific arguments are allowed when supported by MaterialsFramework.

5.3 [tdb_fit], [tdb_sqs], and [tdb_scraps]

[`tdb_fit`] controls relaxation/fitting: `fmax`, `verbose`, `t_min`, `t_max`, `sro`, `bv`, `phonon`, `open_calphad`, and `track_trajectory`. Disabling trajectory tracking reduces storage.

[**tdb_sqs**] controls the mcsqs run time, cutoff interpretation, pair/triplet/quadruplet cutoffs, correlation weights **wr**, **wn**, and **wd**, plus the parallel run count. Cutoff meaning depends on **cutoff_mode**; record the mode with numeric values.

[**tdb_scraps**] controls SCRAPS ranks, automatic search budget, maximum shell, and multibasis sublattice handling.

Each [[**tdb_sqs_levels**]] has a numeric **level**, one or more **compositions**, and the sublattice **letter** list to which the composition applies.

5.4 [**phase**]

Setting	Meaning
key	Phase definition key referenced by the driver.
generator_name	Base phase/prototype name.
a,b,c and angles	Initial cell parameters and angles.
supercell_size	Integer replication along the three lattice directions.
use_average_lattice	Average active elements' lattice references for the phase.
vectors	Three lattice vectors in the prototype convention.
coords	Fractional coordinates followed by variable-site letters or fixed element symbols.

When averaging is enabled, active elements come from the configured phase site definitions or phase element pools. Prototype presets may be used when available; otherwise periodic-table reference cells are used. Missing bulk reference cells fall back to an isotropic covalent-radius estimate. Explicit phase-level or configured element values can override inferred references. Always record the resulting cell values because these are initial structure estimates, not universal experimental constants.

5.5 [**tdb_inputs**]

terms_in and **mult_in** are tables keyed by phase name, optionally including system-size suffixes such as **PHASE_2** or **PHASE_3**. Resolution selects the most specific applicable entry. **sqs_gen_in** can similarly override generated SQS composition text.

Site tables define elements and behavior per sublattice:

```
[tdb_inputs.sites.PHASE.a]
elements = ["A", "B", "C"]
constant = false

[tdb_inputs.sites.PHASE.b]
elements = ["D", "E"]
constant = true
composition = [0.4, 0.6]
```

A constant site preserves the listed element order and fixed composition instead of permuting it. System-specific override tables can replace terms, multiplicities, sites, SQS text, or fixed compositions for a selected chemical system. Fractions on a fixed site must be numeric, match the element count, and sum to one.

5.6 [phase_plots] and [phase_grid]

[phase_plots] specifies input/output folders, plot temperature range, missing-calculation policy, GIF settings, fixed species, variable-site fraction, composition-grid density, convex-hull energy threshold, and pressure. **selected_systems** is inferred from configured TDB element pools unless explicitly implemented by the calling context.

[phase_grid] specifies the GIF directory, panel dimensions, optional column count, output frame rate, background color, header/colorbar dimensions, and temperature range. Its range and step must match GIF frame generation for strict synchronization.

5.7 [database]

This section defines variable and fixed elements, oxidizing species, element count limits, Materials Project stability filtering, duplicate formula handling, POSCAR/energy cache behavior, MLIP name/label/-folder, source composition folders, four database substage switches, and **api_key_env**. Prefer naming an environment variable, normally **MP_API_KEY**. An explicit **api_key** value is also accepted. For compatibility, an unset nondefault value in **api_key_env** is treated as a literal key, but committed credentials should be avoided.

[database_mlipo_kwargs] configures the reference-structure calculator. [database_fallback_refs] provides explicit elemental energy references when the selected MLIP source does not provide one. Mixing inconsistent reference methods changes formation energies and must be treated as a scientific input choice.

5.8 [oxidation] and Modes

[oxidation] selects "batch" or "single", identifies an optional fixed phase element and stoichiometry, chooses structure/database subdirectories, configures region labels, and defines composition-slice axis priority. Use no phase element and zero stoichiometry for a metal-only phase.

[oxidation_batch] independently enables oxidation scans, chemical-potential/composition maps, onset calculations, 3-D onset surfaces, animations, composition slices, and per-composition slice μO -T maps. The **run_calculations** and **run_plots** switches control two global execution passes. When both are true, every selected system is calculated and cached before plotting begins. Calculation-only mode disables all figures and animations; plot-only mode performs no LP solves and reports any missing required cache. A batch with failed systems prints the complete failure summary and exits before a plotting pass can begin. **elements** is an optional allowed metal pool; an empty list includes all systems, while a populated list skips systems containing any other metal. The section also defines cache switches; temperature, chemical-potential, and composition ranges; slice ratios; onset composition resolution; and onset threshold. **slice_muT_comp_step** controls only the fixed compositions used for per-composition μO -T maps; it does not change μO -x slices or onset composition grids. Batch μO -T maps are written only within composition slices. Their region keys contain phase-fraction ranges, and their HTML files report exact pointwise phase fractions on hover. For each composition and temperature, an onset chemical-potential sweep stops at the first infeasible assemblage. The onset table records both the last feasible and first infeasible chemical potential. If no onset occurs over an otherwise valid sweep, downstream AUC ranking uses the last feasible value, not the configured upper chemical-potential bound. The same sweep integrates raw LP fractions for the parent multicomponent phase and for oxygen-containing phases. Parent-phase composition may vary or split across multiple equilibrium pseudophases; all such fractions count while each configured variable element remains above 0.01. End members missing an

element do not count toward the parent-phase integral. The private ranking utility then integrates both quantities over temperature and plots parent-phase retention against oxide formation.

[`oxidation_single`] defines one system, metal ordering, normalized initial composition, temperature and chemical-potential grids, scan temperatures, composition coordinate range, cache setting, component-display setting, Redlich–Kister order, and secondary map step. Its `run_calculations` and `run_plots` switches use the same calculation-first behavior as batch mode.

Ranges are three-value inclusive specifications [`start, stop, step`]. The runner validates positive steps and materializes them with a small floating-point endpoint allowance.

6 Structure and TDB Calculations

6.1 Composition Enumeration

`BladeCompositions` combines primary and secondary pools subject to minimum and maximum counts. System identity is based on valid, unique chemical symbols. The order-independent system key is generated from sorted element symbols, while explicit sublattice order is retained where thermodynamically meaningful.

6.2 SQS Generation

`BladeSQS` creates prototype and SQS composition inputs, asks ATAT to materialize level directories, computes correlations, and launches one or more `mcsqs` searches. Candidate quality is represented by retained correlation outputs such as `bestcorr.out`. Pure-species directories may not need an `mcsqs` search; their treatment is separate from a missing failed search.

`ScrapsSQSGen` provides the alternate SCRAPS path. Backend selection should not change the downstream API, but it can change the selected structures and therefore all fitted outputs.

6.3 MLIP Relaxation and TDB Fit

`BladeTDBGen` is configured without side effects. Calling `fit()` iterates systems and phase data, prepares fitting directories, invokes the `MaterialsFramework` fitting path, and writes TDBs. Calculator choice and calculator arguments are forwarded explicitly; no calculator name is hard-coded in the generalized class.

Refitting is safe with respect to avoiding a new MLIP calculation only when the retained structures and energy files are valid and the intended changes are limited to fitting/model inputs. It can change TDB coefficients and thermodynamic predictions, as expected, while leaving source energies fixed.

7 Phase Visualization

`examples/extra/phase_visualization.py` is the single implementation for phase plotting and synchronized grid compilation. Direct execution uses:

```
run_phase_plots = True
run_phase_grid = True
```

The switches allow plot-only, grid-only, both, or neither. Plotting discovers top-level TDBs under selected system folders. Binary systems receive Gibbs energy, Gibbs mixing, and applicable phase-diagram plots. Higher-order systems use a simplex composition grid, compute mixing free energy, identify the lower convex hull, classify points above the hull threshold, and project the N-component simplex to a regular polygon for display. Temperature frames can be cached and assembled into GIFs.

Grid compilation discovers clean phase-evolution GIFs, groups them by parsed element count, resizes panels, synchronizes frames to the configured temperature array, adds temperature/colorbar context, and streams raw frames to FFmpeg. If no GIF exists, grid-only mode reports the condition and returns normally.

8 Oxidation Database Workflow

8.1 Composition List

`OxideCompositions` builds chemical systems from variable pools, fixed phase species, and the oxidizing species. Inclusion flags control base, oxidized, fixed-species, and no-oxygen combinations. It writes a composition workbook used by reference acquisition.

8.2 Reference Structures and Energies

The Materials Project stage searches each chemical system, optionally limits results to stable entries, skips formulas already represented by BLADE data, and stores POSCARs with metadata. The MLIP stage uses one configured calculator to relax/reference these structures and writes total and per-atom energies.

8.3 Database Assembly

`OxideDatabase` scans BLADE structures and TDB outputs, combines MLIP sources and fallback elemental references, writes unified and per-system energy tables, and collects the files needed by equilibrium analysis into `Files/system_structures/<system>/`. System detection derives from configured directories and formulas; no phase element should be appended implicitly to a system name.

9 Oxidation Equilibrium and Maps

9.1 Public API

`OxidationConfig` carries shared paths, phase species/stoichiometry, labels, and slice priorities. `OxidationCalculator` method `run_batch()` discovers eligible systems and launches generalized analysis. `single()` returns a fixed-composition analyzer for the requested system, metals, and composition. `OxidationVisualizer` exposes visualization behavior separately from calculation setup.

9.2 Equilibrium Model

At each temperature, oxygen chemical potential, and initial composition, the framework constructs the feasible phase set and solves the phase-fraction optimization subject to element mass balance and nonnegative fractions. Consecutive chemical-potential solves are neighboring points in a scan; their

problem structure and cached thermodynamic data can be reused even though each point represents a distinct equilibrium condition.

The configured initial composition belongs to the point being calculated. In composition maps, each grid point is a different initial composition. A guide line should appear on an onset plot only when the complete diagram is defined relative to that initial composition.

9.3 Region Identity and Labels

Region separation is based on the set of displayed compounds after the configured composition-component cutoff is applied. Fraction-range changes alone do not create a separate region. If a component enters or leaves the displayed phase composition at the cutoff, the displayed compound identity changes and a new region may form.

Static keys summarize phase-fraction and phase-composition ranges. Endpoint values near zero or one can be represented as 0 or 1 according to configured labeling rules; small composition components can be omitted or included according to the relevant option. Exact HTML hover text is generated from the underlying point, not the summarized region key. Values below 0.01 retain additional significant decimal places in hover text; ordinary values are shown compactly.

9.4 Outputs

Depending on switches and system size, outputs include:

- oxidation scan curves and per-temperature folders;
- oxygen-chemical-potential/composition assemblage maps;
- per-composition oxygen-chemical-potential/temperature slice maps;
- miscibility-gap panels and composition-slice maps;
- onset curves, AUC summaries, and 3-D onset surfaces;
- equilibrium phase-composition 3-D plots;
- GIF/MP4 animations;
- interactive HTML plots with exact point hover values;
- per-composition tables and compact NumPy cache files.

Composition-slice output uses a selected axis and configured remainder ratios. Axis priority selects the first applicable element; it avoids generating every permutation when only one ordered axis family is desired. Static μO -T maps are stored in each slice's `muO_T_plots/` directory; GIF/MP4 and other aggregate slice products remain at the slice root.

10 Caching, Restarting, and Storage

Artifact/setting	Reuse behavior
<code>bestcorr.out</code> and SQS outputs	Allow completed SQS searches to be skipped. Validate parseability; existence alone may not prove a valid result.
<code>skip_existing_tdb</code>	Avoids a complete existing TDB fit unless refit is requested.
<code>refit_existing_tdb</code>	Reuses relaxed energies but refreshes fitting inputs and TDB parameters.
<code>skip_existing_plots</code>	Checks individual enabled plot files so missing products can be generated independently.

Artifact/setting	Reuse behavior
<code>skip_if_tables_exist</code>	Reuses matching equilibrium tables instead of repeating solves.
<code>skip_if_analysis_exists</code>	Reuses derived oxidation analysis products.
<code>.npz</code> caches	Compact numerical data used for redraw/interactive output.
Trajectory setting	Disable trajectory generation to reduce storage if no relaxation movie is needed.

Do not reuse a cache after changing solver grids, source TDBs, phase energies, element ordering, fixed species, initial compositions, model terms, or other inputs that affect the cached quantity. Removing only a PNG or HTML file should not require recalculation when the corresponding numerical cache is intact.

11 Every Example Script

Script	Role and direct-run behavior
<code>examples/full_framework.py</code>	TOML orchestrator; defaults to the adjacent standard TOML. Supports <code>--check</code> and <code>--dry-run</code> .
<code>structures/01_compositions.py</code>	Standalone composition enumeration; no generated input required.
<code>structures/02_sqs_generation.py</code>	Standalone ATAT SQS generation from in-file prototype settings.
<code>structures/03_tdb_generation.py</code>	Standalone TDB fitting from existing SQS data.
<code>structures/04_visualization.py</code>	Standalone examples for structures, energies, phase diagrams, and movies; skips some absent inputs.
<code>structures/05_data_analysis.py</code>	Standalone scan of structure/energy directories into tabular data.
<code>structures/tdb_gen_hedb.py</code>	Complete standard/mcsqs prototype demonstration and orchestrator driver.
<code>structures/tdb_gen_hedb_scraps.py</code>	Complete standard/SCRAPS demonstration and driver.
<code>structures/tdb_gen_max.py</code>	Complete multibasis/mcsqs demonstration and driver.
<code>structures/tdb_gen_max_scraps.py</code>	Complete multibasis/SCRAPS demonstration and driver.
<code>oxidation/01_compositions.py</code>	Standalone oxidation-system enumeration.
<code>oxidation/02_poscars.py</code>	Standalone Materials Project acquisition; requires the step-01 workbook and a key.
<code>oxidation/03_energy.py</code>	Standalone reference-structure MLIP relaxation.
<code>oxidation/04_database.py</code>	Standalone database assembly and structure collection.
<code>oxidation/database_framework.py</code>	Resumable class-based combination of oxidation steps 01–04 using in-file settings.
<code>oxidation/oxidation_framework.py</code>	Standalone batch/single oxidation analysis using in-file settings.
<code>extra/phase_visualization.py</code>	Combined phase plotting/GIF and grid-video implementation with two booleans.
<code>extra/compare_energy.py</code>	Energy-source parity, residual, and tabular comparison.

Script	Role and direct-run behavior
<code>extra/refit_tdb.py</code>	Refit TDB model terms from existing structures and energies.
<code>extra/tdb_gen_alloy.py</code>	Complete single-sublattice alloy demonstration.
<code>extra/tdb_gen_hedb_quaternary.py</code>	Complete higher-order prototype demonstration.

Every example Python file is directly executable, but later workflow steps require the artifacts or services stated above. Library files under `src/blade/` are imported APIs and are not intended as standalone commands.

12 Library Class and Module Reference

12.1 `blade.tools`

- `blade_compositions.py`: `BladeCompositions` generates constrained chemical systems.
- `blade_sqsgen.py`: `BladeSQS` creates ATAT SQS inputs and executes SQS generation.
- `blade_scraps_gen.py`: `ScrapsSQSGen` provides the SCRAPS backend.
- `blade_tdb_gen.py`: `BladeTDBGen` prepares and runs structure-energy/TDB fitting.
- `blade_cutoff.py`: neighbor/correlation cutoff support.

12.2 `blade.analysis`

- `blade_data.py`: `BLADEDData` extracts structural and energy information into data tables.
- `blade_visual.py`: `BladeVisualizer` produces structure images, Gibbs curves, phase diagrams, composites, and relaxation movies.

12.3 `blade.oxidation`

- `compositions.py`: `OxideCompositions`.
- `database.py`: `OxideDatabase`.
- `oxidation_calculations.py`: public `OxidationConfig` and `OxidationCalculator`.
- `oxidation_visualization.py`: public `OxidationVisualizer`.

12.4 `blade.oxidation.framework`

Module	Responsibility
<code>config.py</code>	Shared analysis options and path/range configuration.
<code>system_config.py</code>	System discovery, metal detection, phase/TDB path assembly.
<code>thermodynamics.py</code>	Phase thermodynamic data and equilibrium/optimization support.
<code>compiler.py</code>	Input-table and phase-data compilation.
<code>analyzer.py</code>	Main analysis orchestration, region construction, labels, and plot dispatch.
<code>maps.py</code>	Composition and chemical-potential map calculations.
<code>single.py</code>	Fixed-composition workflow.

Module	Responsibility
<code>batch.py</code> and <code>_batch_</code>	Multi-system batch workflow.
<code>internal.py</code>	
<code>_run.py</code>	Shared run entry behavior.
<code>utils.py</code>	Parsing, formatting, and numerical utilities.

13 Reproducible Execution Records

For every scientific run, archive:

1. the exact TOML or direct script;
2. BLADE commit hash and uncommitted diff;
3. `uv.lock` or `pixi.lock`;
4. Python, external binary, MLIP, and model versions;
5. element pools/order, phase sites, cell, supercell, and fixed species;
6. SQS backend, levels, cutoffs, budgets, seeds where available, and selected correlation output;
7. relaxed structures, energies, reference energies, and fitting files;
8. generated TDBs and phase database inventory;
9. temperature, chemical-potential, and composition grids;
10. cache settings and a manifest of reused artifacts;
11. output tables, NumPy caches, and figures.

Recommended command record:

```
git rev-parse HEAD
git status --short
pixi run python examples/full_framework.py --check
pixi run python examples/full_framework.py --dry-run
pixi run python examples/full_framework.py 2>&1 | tee Files/run.log
```

14 Quality Checks

```
# Repository lint and format verification
pixi run check

# Explicit Python compilation
pixi run python -m compileall -q src examples

# Configuration/dependency validation
pixi run python examples/full_framework.py --check

# Stage-routing validation without calculations
pixi run python examples/full_framework.py --dry-run

# uv lock and uv environment validation
uv lock --check
uv run --locked --extra workflow python examples/full_framework.py --check
```

To format edited Python:

```

pixi run isort src examples
pixi run black src examples
pixi run ruff check src examples

```

15 Troubleshooting

Symptom	Interpretation and action
Nosuchfileordirectory: .../grace	A calculator-specific folder/path was selected by configuration or old generated data. Set the calculator and all base paths in the direct script/TOML; generalized library code should not infer a calculator-named folder.
metaldetection... positionalargument	Caller/method signature mismatch. Validate with the current package and runner versions; do not mix copied framework revisions.
expected2+metals	System detection found fewer variable species after fixed species were removed. Verify phase coordinates, fixed-element configuration, and directory/formula names.
Nobestcorr.out	Pure species may legitimately skip mcsqs; otherwise the SQS search is incomplete.
Couldnotparseobjective	The file exists but is not a valid completed correlation result. Do not treat it as a reusable completed SQS.
NoTDBfilesfound	TDB fitting did not run, selected output folder differs, or system filtering excludes the files. Check stage switches and <code>comps_folder</code> .
expectedtwometals	A binary-only Gibbs/phase plot received a higher-order system. Higher-order phase-map logic should handle those systems instead.
needatleastonearray toconcatenate	picalphad found no tie-line data for the requested binary plot. Treat as a plot-specific failure; retained energy plots/TDB may still be valid.
NocleanGIFsfound	Grid compilation has no cached animation input. Enable phase plotting/GIF creation or point <code>phase_diagrams_folder</code> to existing GIFs.
TOMLDecodeError: Cannotoverwriteavalue	The same key was assigned twice in one TOML scope. Remove or rename the duplicate assignment.
Plots recalculate unexpectedly	Preserve equilibrium tables/. <code>npz</code> , enable cache switches, and verify that grid/settings and output paths match the original run.
Missing plot families	Check both major stage switches and plot-specific booleans; a cache skip should be evaluated per output, not per entire plot group.

16 Extending to a New Material System

1. Copy a TOML to a new, versioned configuration.

2. Define primary/secondary element pools and desired system sizes.
3. Define the phase cell, vectors, coordinates, and supercell.
4. Mark variable sites with site letters and fixed sites with actual species; do not add a demonstration fixed element unless it is physical.
5. Define site element lists, constants, and compositions.
6. Define SQS levels and backend settings.
7. Set calculator and relaxation/fitting settings.
8. Run `--check`, then SQS only.
9. Inspect structures and correlation objectives.
10. Run TDB fitting/refitting and inspect fit/phase outputs.
11. Define database species/reference sources and construct the collected system database.
12. Define oxidation phase species, grids, mode, and output switches.
13. Run a reduced validation grid before the full unchanged-resolution production run.
14. Archive all provenance listed above.

17 Standalone Versus Configured Execution

`phase_visualization.py`, numbered examples, complete prototype drivers, `database_framework.py`, and `oxidation_framework.py` can run with their in-file settings and no TOML. `full_framework.py` is intentionally TOML-driven, but its argument is optional because it defaults to `examples/full_framework.toml`. Package modules are intended to be imported into user code:

```
from blade.tools import BladeCompositions, BladeSQS, BladeTDBGen
from blade.analysis import BLADEDData, BladeVisualizer
from blade.oxidation import OxidationCalculator, OxidationConfig
```

The README files under `examples/` provide the shortest retraceable run sequence. This manual provides the full reference and should be updated when entry points, settings, output layout, or scientific assumptions change.